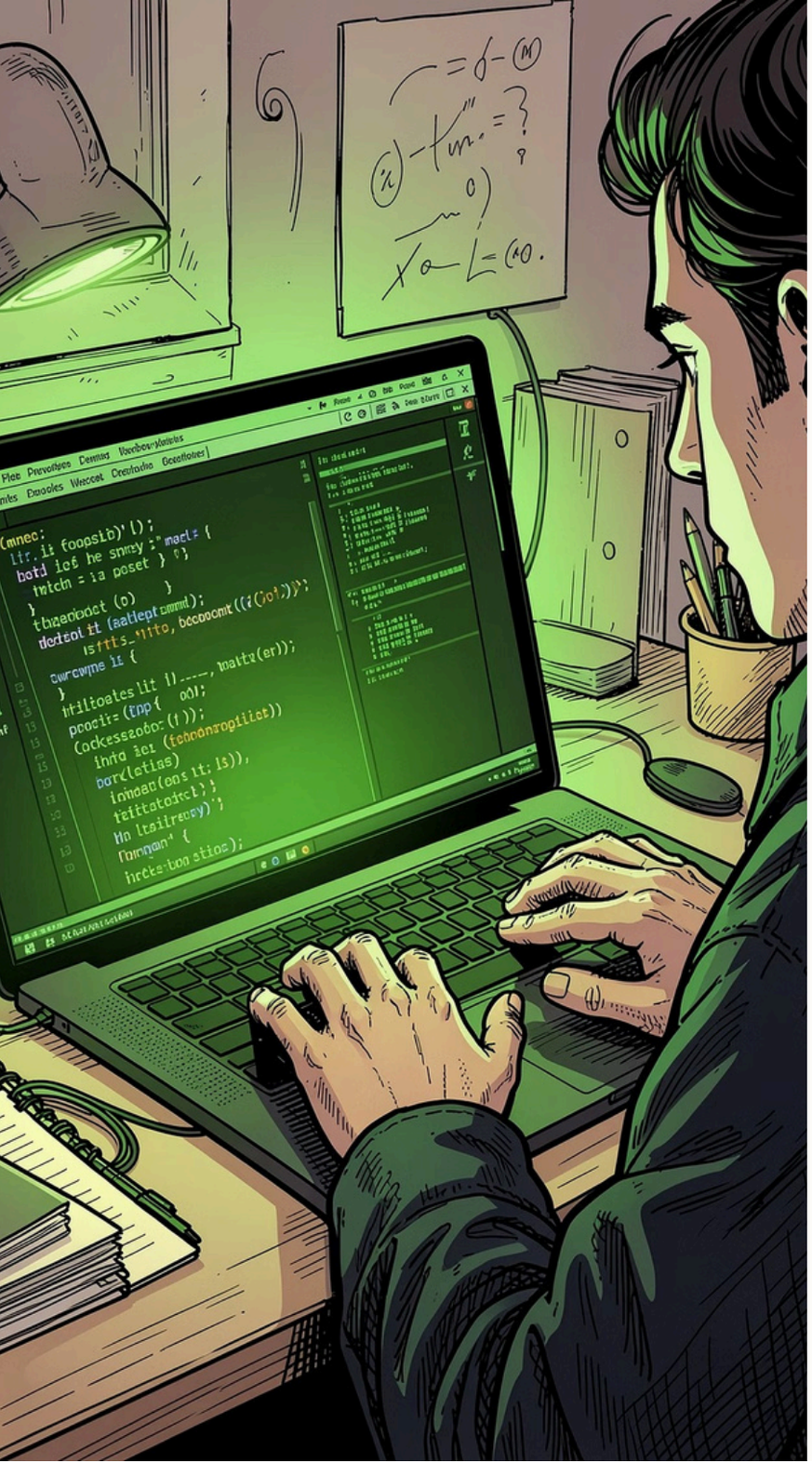
Project 2: Basic Encryption & Decryption

Caesar Cipher Implementation

Devki Prajapati | Cybersecurity Intern | DecodeLabs Batch 2026



Project Overview

Goal: EncryptanddecryptusertextusingCaesar Cipher.

1	**Encrypt Text** Apply shift-based logic to transform plaintext into ciphertext
2	**Decrypt Back** Reverse the shift to recover the original message
3	**Handle All Inputs** Spaces, numbers, and punctuation preserved as-is
4	**Show All Outputs** Display Original, Encrypted, and Decrypted results

Tools Used: Python 3, VS Code

HISTORY

What is Caesar Cipher?

The**oldestencryption technique** — famously usedbyJulius Caesarto protect military communications.

Shift = 3 Example

A → D, B → E, H → K, Z → C (wraps around the alphabet)

Symmetric Encryption

The same key encrypts *and* decrypts — simple and elegant

The Math Behind the Cipher

Encryption Formula

$$E(x)=(x+n)\mod 26$$

Decryption Formula

$$D(x)=(x-n)\mod 26$$

Where **x** = character position (0–25) and **n** = shift key.

Python Conversion Functions

ord()

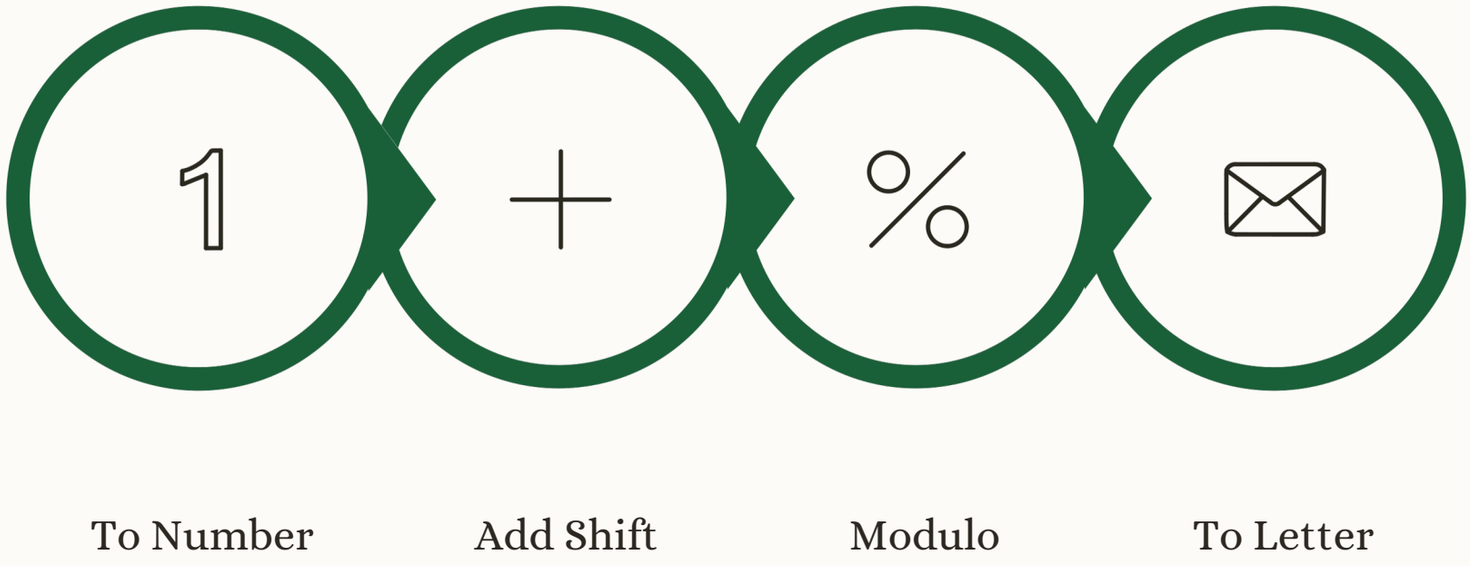
Converts a letter to its ASCII numeric value

chr()

Converts a number back to its character representation

Algorithm: Step by Step

Encrypting the letter "**A**" with a shift of **3**:



The diagram above illustrates how each letter is converted to a numeric position, shifted, wrapped using modulo, and converted back to a character.

Wrap-Around Example

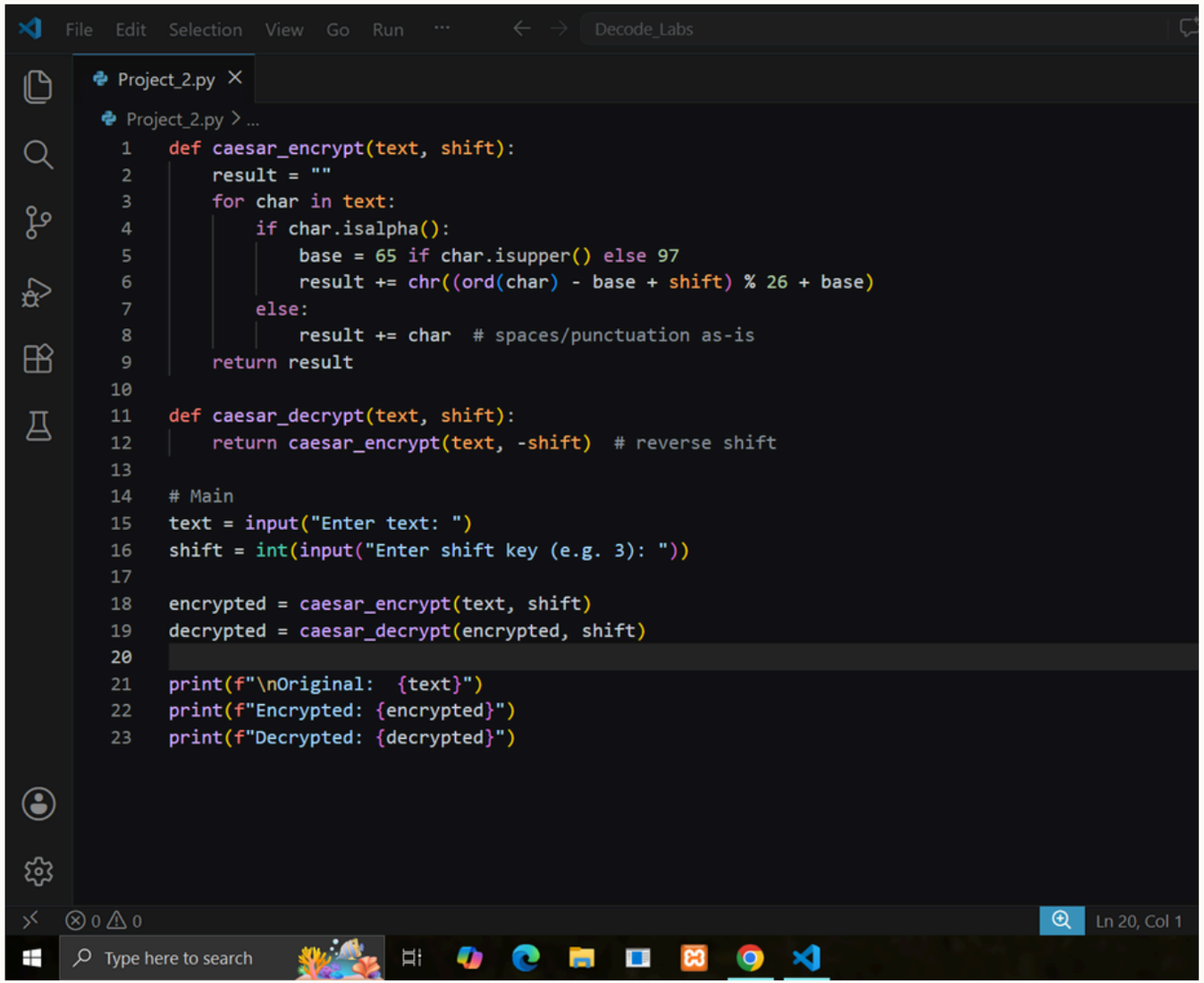
Encrypting **Y** with shift 3:

- Y is at position 24
- 24 + 3 = 27
- 27 % 26 = **1** → **B**

> The **% 26** modulo operation is what makes the alphabet wrap seamlessly from Z back to A.

PYTHON IMPLEMENTATION

Code Breakdown



caesar_encrypt()

Applies the forward shift (+n) to each alphabetic character

caesar_decrypt()

Applies the reverse shift (-n) to restore original text

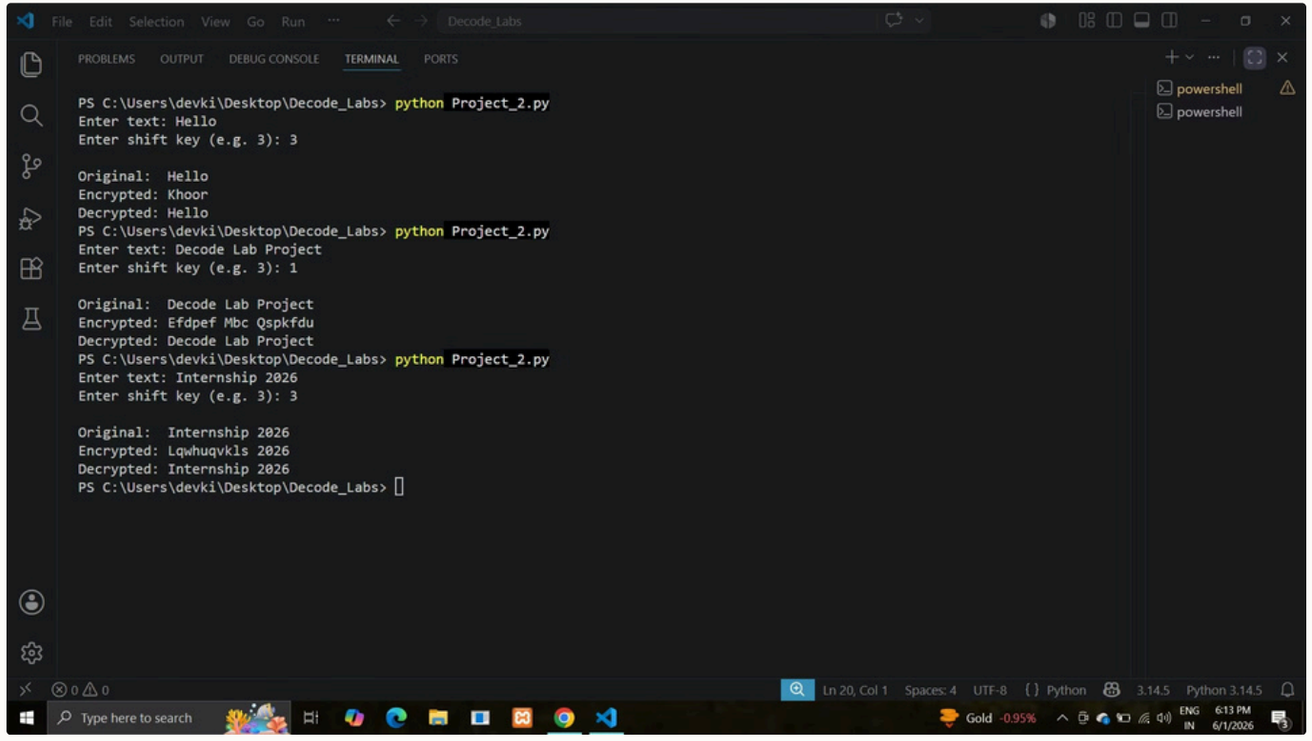
isalpha() check

Skips spaces, numbers, and symbols — copies them as-is

isupper() handling

Treats uppercase (base 65) and lowercase (base 97) separately

Output Proof



Test Input	Shift	Encrypted Output	Decrypted Output
"Hello"	3	Khoor	Hello ✓
"Decode Lab Project"	1	Efdpef Mbc Qspkfdu	Decode Lab Project ✓
"Internship 2026"	3	Lqwhuqvkls 2026	Internship 2026 ✓

Edge Cases Handled

Uppercase vs Lowercase

Base 65 (A–Z) and base 97 (a–z) used separately to preserve case

Wrap Around Z→A

% 26 modulo handles automatic wrap from the end of the alphabet back to the start

Spaces & Numbers

isalpha() check ensures non-letter characters are copied exactly as-is

Custom Shift Key

User inputs their own key every time — fully flexible encryption strength

Vulnerability & Evolution

⚠️ Caesar Cipher Weakness

Only **25 possible keys** — trivial to brute force in seconds

Frequency analysis can crack it without even trying all keys

🔒 Real-World Standard: AES

AES 256-bit

Confusion + Diffusion + 128-bit keys

Caesar = Foundation

Teaches the core logic of substi tution

AES = Real Shield

Protects real-world data at scale

Learnings & Thank You

Technical Skills

- ASCII encoding & character mapping
- Modular arithmetic in practice
- Function design & reusability
- Edge case handling

Cybersecurity Concepts

- Data Confidentiality
- Symmetric Encryption
- IPO Model (Input → Process → Output)

Project 2 Successfully Completed! — Devki Prajapati | DecodeLabs Batch 2026